

Intro to Local LLMs



Kei Tang

2026-09-26

Recurse Center



1. Introduction	2
2. Model sizes and memory: What model can I run?	6
3. GPUs and optimizations: How fast will the model run?	13
4. Model repositories and Inference frameworks	18
5. Demo	21

1. Introduction





- **Privacy:** Cloud providers may retain your data or use it for training
- **Cost:** Powerful cloud models are expensive
- **Research and learning:** Know more about how LLMs work
- **Ethics:** Avoid the big model providers controlling the technology
- and more?

1.2 Local LLMs are quite powerful now

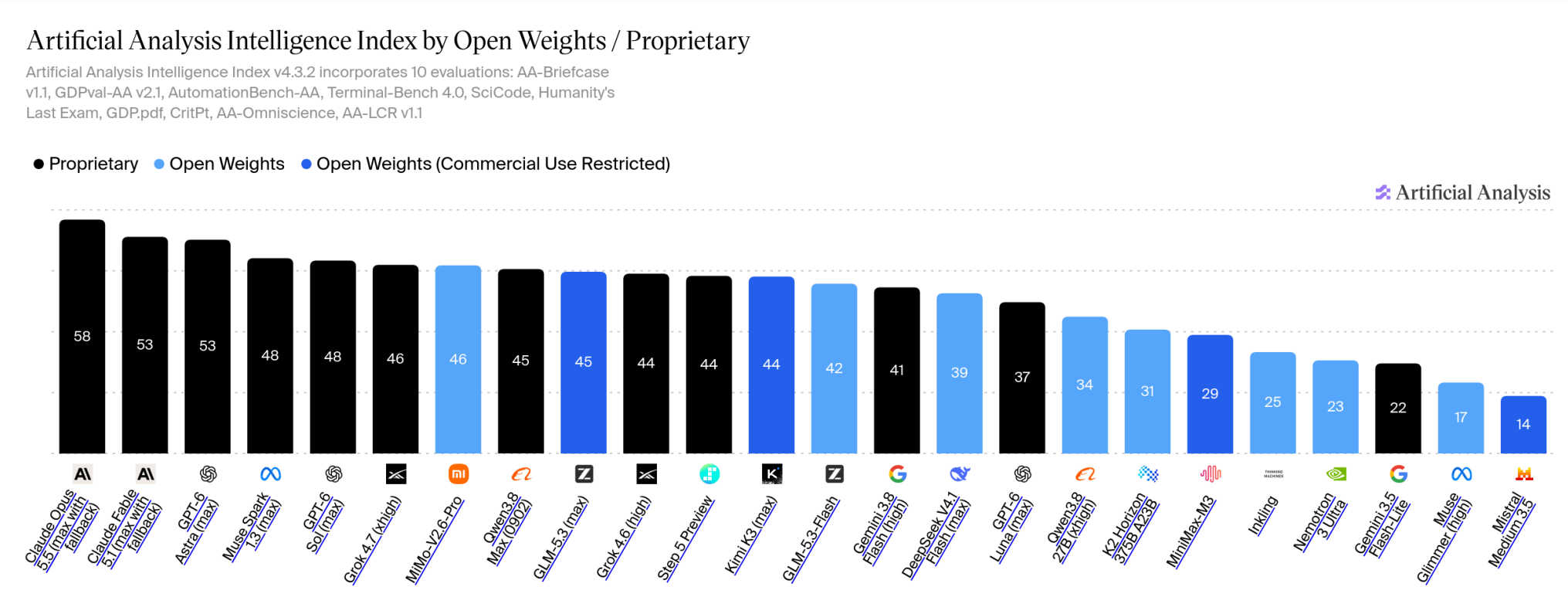


Figure 1: The AA Intelligence Index. (Take all benchmark rankings with a grain of salt!)



- **Harness**: Models are generally decoupled from harnesses
- **Training**: Maybe another workshop in the future?
- **Platform specific settings (e.g. Metal for Macs, Nvidia stuff)**: I don't own those hardware, but we can look into it together!

2. Model sizes and memory: What model can I run?



Model size for language models are usually characterized by **number of parameters** (in billions):

- The scaling era: GPT-2: 1.5B, GPT-3: 175B, GPT-4: 1.8T (unconfirmed)
- Best open models: ~1T
- With consumer hardware, 100B is usually the limit (...Why?)



Traditionally LLMs are deployed on GPUs, so the model needs to fit on the VRAM.

- **Exceptions:**

- ▶ Newer Macs, some AMD chipsets, (and game consoles) use *unified memory*, which means that system RAM and VRAM are shared.
- ▶ *GGUF* model format makes offloading part of the model in system RAM easy, albeit with a small performance overhead.

Practically, “memory” refers to system RAM + VRAM.



- **Model parameters:** $\text{memory} = \text{number of parameters} * \text{bits per parameter}$
 - By default, LLMs use FP16 (16bit). A 100B model will consume $100\text{B} * 16\text{bit} = 200\text{GB}$.
- **Dynamic memory usage:** Additional memory is required to store intermediate computation values (mostly KV cache).
 - This part *grows with context length*. When a large context length is required, this extra memory can be as large as $0.5 * \text{model size}$.



Quantization takes an existing model, reduces memory usage by changing the numeric data structure for parameters, usually with little drop in accuracy.

- Classic quantization methods use INT8 or INT4.
- Recent methods dynamically select some parameters to quantize in order to minimize accuracy loss. *Quants* range from 1 to 8 bits.
- eg. $100\text{B} * 4\text{bit} = 50\text{GB}$

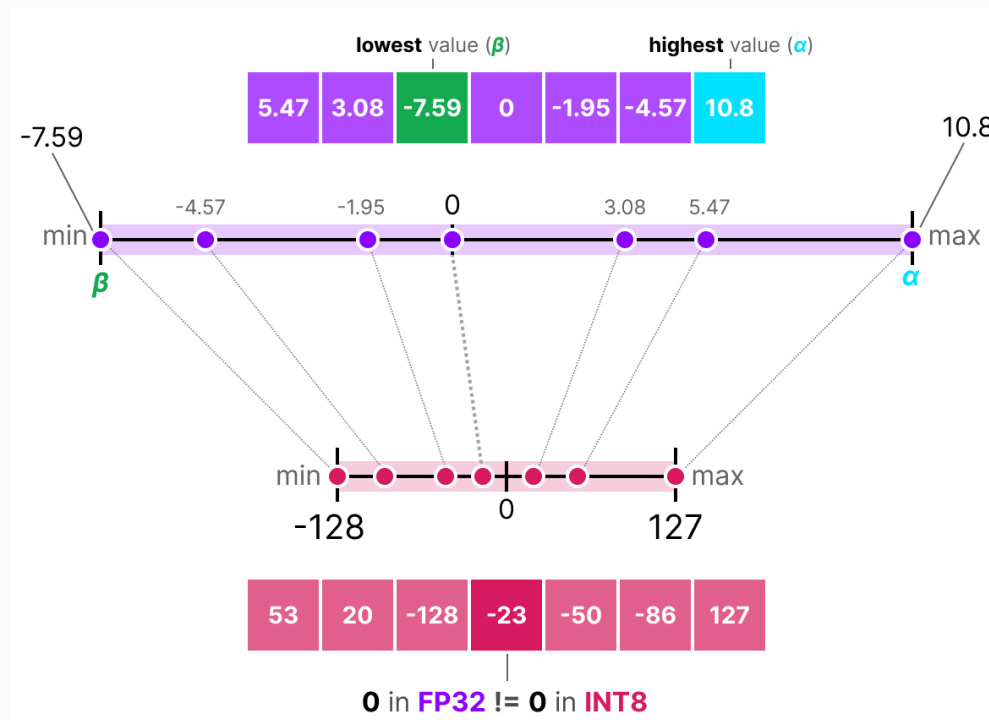


Figure 2: From FP32 to INT8¹

¹Image credit: <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

2.4 Quantization

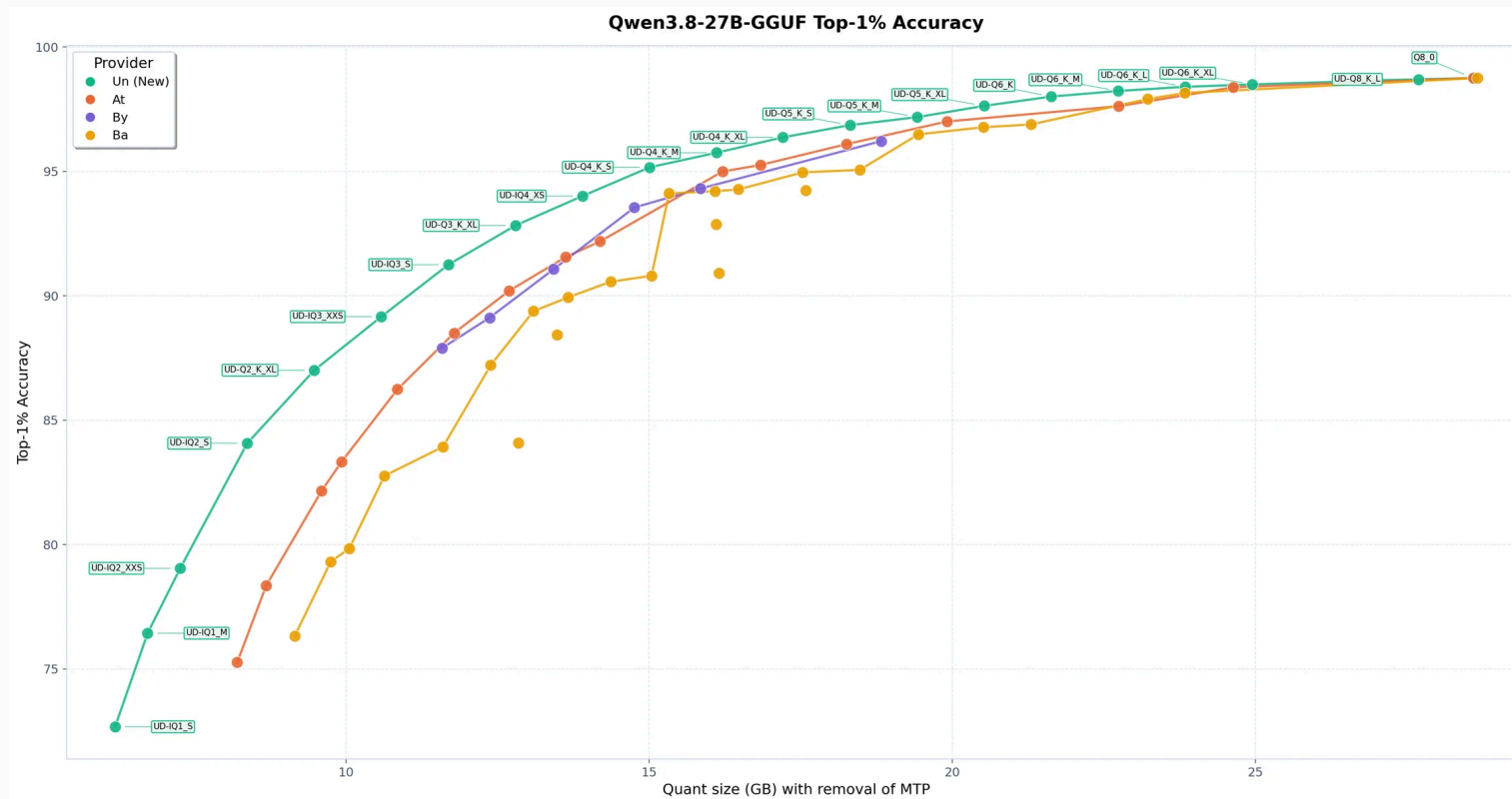


Figure 3: The different Qwen3.8-27B quants (X: model size, Y: Top-1 accuracy)¹

¹<https://unsloth.ai/docs/basics/dynamic-3.0-ggufs>



As a general rule of thumb:

- Memory (in GB) $\sim$ the size (in B) of model you can run comfortably when quantized

3. GPUs and optimizations: How fast will the model run?

3.1 How “speed” is measured



One call to the LLM involves two main steps:

1. **Prefill/Prompt evaluation**: The model processes the whole prompt/context in one pass.
2. **Generation**: The model generates the output tokens one by one.

The speed for each step can be measured by **token per second**.

- Prefill is much faster because it's parallel
- Which one is more important depends on use case



- **Model size:** Larger model -> More computations and more data to move around
- **GPU processing power:** Usually measured in TFLOPS (floating point ops per second)
- **Quantization:** Lower quantization bits -> generally faster
- **Model features:** MoE, MTP...
- Others:
 - Underlying framework & drivers
 - Platform-specific optimizations

3.3 Mixture of Experts (MoE)



Mixture of Experts models contain many different sub-models (“experts”), and during inference only one “expert” is activated.¹

- Memory requirement is still high, but inference speed is much faster.
- e.g. Gemma - 4 - 26B - A4B has 26B parameters, but only 4B is activated at a time.
- Usually slightly *less capable* compared to “dense” models at same size.

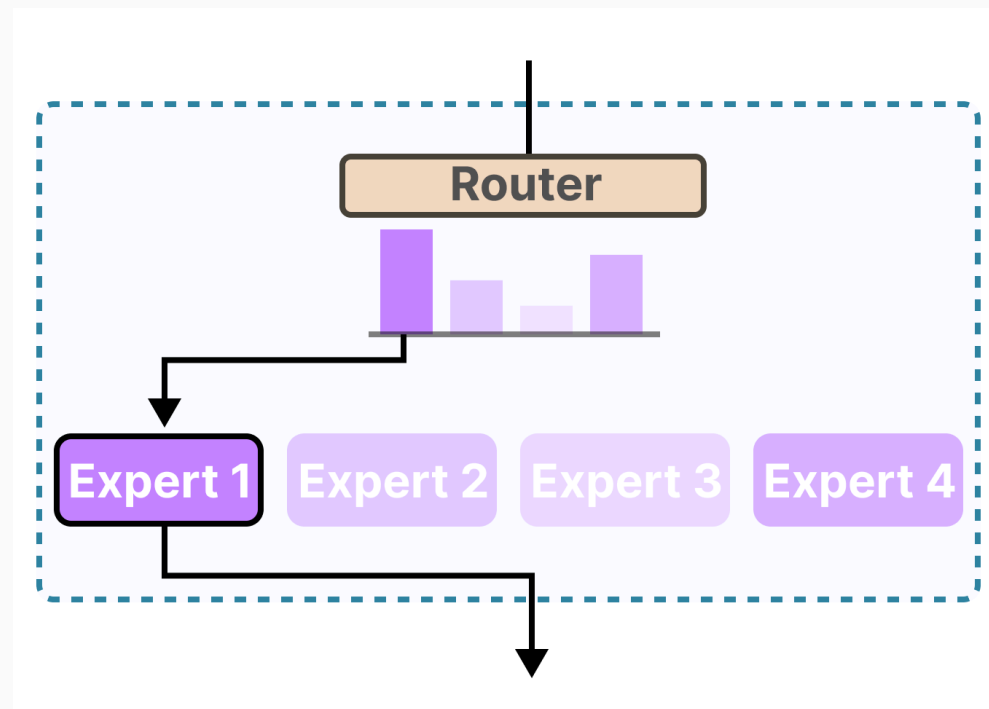


Figure 4: Illustration of MoE²

¹Simplified explanation

²Image credit: <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts>

3.4 Multi-token Prediction (MTP)



Some models support **Multi-token Prediction**, which uses a “*draft and verify*” approach to generate multiple tokens in parallel, with *no accuracy loss*¹.

- If available, turning on can improve generation speed.

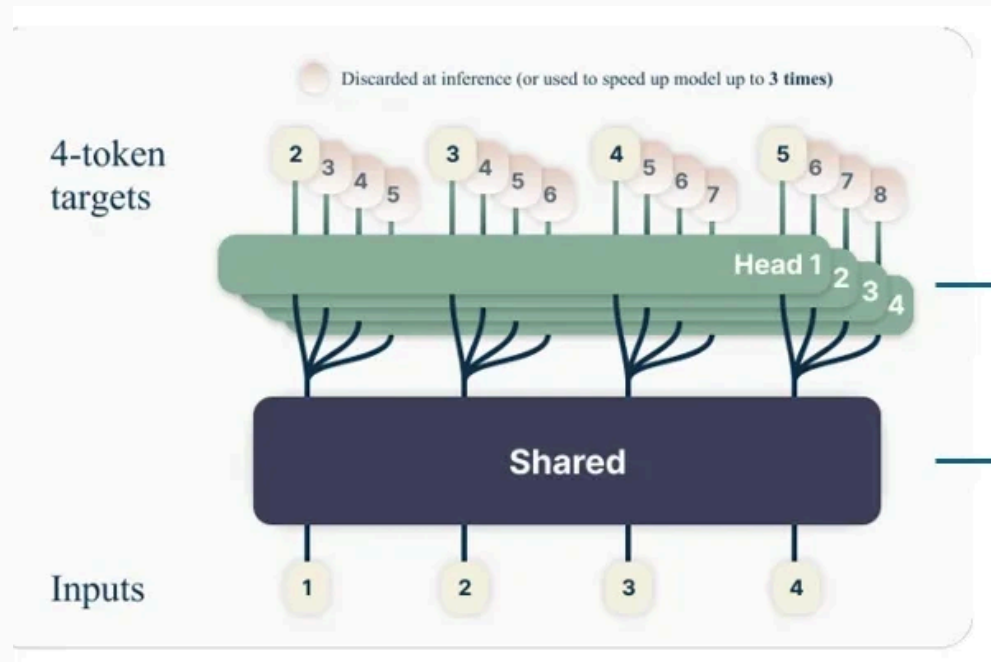


Figure 5: Visualizing MTP²

¹Simplified explanation

²Image credit: <https://medium.com/data-science-collective/deepseek-explained-4-multi-token-prediction-33f11fe2b868>

4. Model repositories and Inference frameworks



- End goal is the same for all of them: An **OpenAI-compatible** API server.
- The main ones: vLLM, SGLang, Ollama, llama.cpp
 - For local deployment with quantized GGUFs, **llama.cpp** is recommended.
 - vLLM or SGLang may be a good choice for Nvidia hardware.
- Easy and powerful solutions:
 - **Ramalama** for people willing to work with containers. Uses vLLM or llama.cpp under the hood, while simplifying the setting up process.
 - **Unloth studio** for people liking GUI. Full selection of models and quants available.



Major model repositories:

- **Huggingface**: <https://huggingface.co/models>
- **Ollama**: <https://ollama.com/library?sort=newest>

For finding quantized GGUFs:

- **Unsloth** provides quantized models for all the latest open source models. It often comes with fixes for the model, and tutorials. *Recommended!*

5. Demo





1. Choosing a model based on the hardware
2. Downloading a quantized GGUF
3. Running it using a inference framework